后训练学习计划 • 第 10 周（Phase 3 开工 • 毕业作品 M4《明日记忆 Tutor 小模型》立项与 SFT 阶段，11/9-11/15）

本周定位：**把前九周的一身本事串成一个真项目**。M4 不是第四个实验，是你第一次以「负责人」身份做一次完整的后训练工程：立项文档写清目标与**可验证的成功标准**，数据复用第 7 周自己发布的数据集，训练方案自己定，**评测协议在训练开始之前冻结**（30 个真实查词场景做 API 版 vs 小模型版的 A/B），然后跑通第一版 SFT，产出 M4 系列博客第 1 篇。

用法说明：每天先看「今日总览」，再从任务 1 逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗 = 硬性要求**，灰色小字块 = 注释（可跳过，不影响主线）。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一	立项：目标 / 范围 / 可验证成功标准	`PROPOSAL.md` 定稿，成功标准每条带数字	6h
Day 2 周二	数据准备：第 7 周数据集体检 + 真实场景脱敏 + 任务格式	train/val/test 切分 + **30 场景封存**	6h
Day 3 周三	训练方案设计 + 0.5B 冒烟	超参表带依据 + 冒烟 loss 下降	6.5h
Day 4 周四	**评测协议先于训练冻结** + API 版基线实测	`ab-protocol.md` + git tag + API 基线三组数	6.5h
Day 5 周五	跑通第一版 SFT（1.5B LoRA）	adapter + eval loss + 30 场景输出留档	7h
Day 6 周六	小模型版 A/B 实测 + 盲评 + 第 1 篇博客	A/B 结果表 + 盲评分 + 博客发布	6h
Day 7 周日	补漏 + 闭卷复测 + 总验收 + 预习 + 求职节点登记	验收 ≥6/7 项通过	4h

注：**本周卡时上限 14h，预估花费 ≈¥30**（4090 24G ¥1.5-2.5/时）。分配：Day 3 冒烟 ≤1h、Day 5 训练约 5h、Day 6 推理实测约 3h，余 5h 给 Day 5 重跑与 Day 6 补测。**API 另计 ≤¥6**（Day 4 跑 30 场景 × 5 次的基线 + 判卷），全周上限 ¥36。7B-Instruct 只做 QLoRA 冲刺项，一旦开跑就砍 Day 6 的重复次数（5→2）来换卡时，**别加预算**。

注：**本周最大风险不是训练，是「目标定得太虚导致整周无法判卷」**。M4 是简历中心项目，最坏的情况不是效果差，是你说不出它「成功在哪、失败在哪」。所以 Day 1 任务 2 逼你给每一条成功标准配一个数字与测量方法，Day 4 又逼你在训练之前把协议钉死（git tag + 文件哈希）。**这两件事是本周真正的产出，SFT 只是第一次执行。**

注：砍单顺序（实在做不完按这个砍）：Day 3 任务 5 的方案文档扩写 → Day 6 的 30 场景重复次数 5→3 → Day 5 的 7B QLoRA 冲刺 → **Day 1 立项、Day 2 封存 30 场景、Day 4 协议冻结、Day 5 第一版 SFT、Day 6 的 A/B 表不许砍**。第 12 周的整体评测与集成会把 Day 6 的粗版补齐，但**协议冻结只有一次机会**，训完再定就是作弊。

二、开始前：环境 / 数据 / 账号（约 40 分钟，周日晚上做完）

1. 本地环境沿用 `.venv-dpo`（上周那个），补两样：

```
cd E:/study/posttrain && source .venv-dpo/Scripts/activate
pip install -U "trl>=0.11" "transformers>=4.45" peft accelerate datasets jsonschema openai \
    -i https://pypi.tuna.tsinghua.edu.cn/simple
python -c "import trl, transformers, torch, jsonschema; print(trl.__version__, transformers.__version__,
torch.__version__)"
```

> 注：`OMP: Error #15` **这周还会来一次**（`pip install -U trl` 可能把 torch 顶成新版本，conda base 与 venv 混用时 OpenMP 又打架）。正解不变：只用那个独立 venv，急着往下做就先 `export KMP_DUPLICATE_LIB_OK=TRUE`（Git Bash）/ `set KMP_DUPLICATE_LIB_OK=TRUE`（cmd）。**torch 版本若被顶掉，`pip install torch==<你上周 torch-pin.txt 里的版本>` 装回去，别去重装 CUDA。**

2. 建目录，确认第 7、8、9 周的产物在位（本周全靠它们）：

```
mkdir -p E:/study/posttrain/week10/{day1,day2,day3,day4,day5,day6,day7}
ls E:/study/posttrain/llm-posttraining-lab/m2-sft/dpo    # 周 8 的 DPO 管线与 pair 脚本
ls E:/study/posttrain/llm-posttraining-lab/m3-grpo       # 周 9 的验证器三件套
# 第 7 周数据集：本地缓存或魔搭/HF 上的 english-tutor-instruct
```

3. 明日记忆侧只读准备（**功能开发冻结 16 周，本周不改产品代码**）：导出查词日志的一个只读快照（CSV/JSONL），字段只要查询词、返回释义长度、时间戳。**用户标识、IP、账号一律不许带出来**——导出的第一步就是**不导这些列**，脱敏在 Day 2 任务 2 做。
4. 账号与余额：AutoDL ≥¥70（本周 ¥30 + 周 11 ¥30 + 余量）；API 余额 ≥¥10（Day 4 基线实测）。HF 打不开走 `export HF_ENDPOINT=https://hf-mirror.com`，或魔搭直下 `modelscope download --model Qwen/Qwen2.5-1.5B --local_dir ./qwen15b`（0.5B 同理）。
5. 上周 gaps 清账：从 `week09/gaps.md` 取三条（其余不动）：①「规则奖励只覆盖 GSM8K，教学场景验证器没真用过」→ 本周 Day 2 任务 3 就用 `reward_cefr` 的格式定 tutor 输出；②「长度偏置只观察到没修」→ 本周 Day 6 任务 2 的盲评里显式打「简洁性」这一轴；③「extract 兜底提取有漏洞」→ 本周不许再用兜底提取，tutor 输出一律走 JSON schema 校验。

Day 1 • 周一：立项文档——目标、范围、可验证的成功标准

今日总览（6h）：今天一行训练代码都不写。产出一份 `PROPOSAL.md`，让**三个月后的你和面试官都能在 5 分钟内判断这个项目「成了没有」**。核心纪律：**每一条成功标准都要带数字和测量方法**，写不出数字的标准就是没标准。

任务 1：一句话定位与「面试官会怎么问」（约 40 分钟）

1. 用一句话写死项目定位（≤40 字），示例结构：「用后训练把明日记忆的 tutor 路径从付费 API 换成可自托管的 1.5B 小模型，在真实查词场景上质量 ≈ API、成本 ≈ 1/20」。

2. 下面列 6 个面试官一定会问的问题，**今天只写问题不写答案**（Day 4/Day 6 的回答会成为答案的一半）：
    - 为什么不用 RAG 或 prompt 工程就够了，非要微调?
    - 为什么是 1.5B 不是 7B？（**答案要能画出延迟/显存/成本三角**）
    - 你的「质量」怎么定义、谁判、判几次?
    - 训练数据从哪来、合规怎么处理?
    - 评测协议是什么时候定的？（**这题就是你今天写这份文档的意义**）
    - 上线替换 API，最坏情况是什么、怎么回滚?

完成判据：`PROPOSAL.md` 第一节有那句话（≤40 字）+ 6 个问题原样列出；第 1 与第 6 题今天就各写 ≥2 行初步答案（其余四题标 `[待 Day X]`）。

> 注：第 1 题要在 RAG/微调之间做过比较：知识频繁更新、需要溯源 → RAG；要改**行为模式与输出格式**、对延迟与成本敏感、要私有部署 → 微调。tutor 属于后者——你要的不是新知识，是**稳定输出一个能被程序解析的教学结构**，这件事 prompt 工程做不干净（Day 4 会用 A 臂的合法率实测把它变成数字）。**这句判断是 M4 立项的合法性来源，别写虚。**

任务 2：可验证的成功标准（约 90 分钟）

在 `PROPOSAL.md` 写这张表。**硬性要求：6 条全部有数字门槛 + 测量方法 + 由哪天/哪个脚本产出。**

#	成功标准	数字门槛（预期量级，Day 4 冻结时定稿）	怎么测	产出在
S1	输出格式合法	JSON schema 校验通过率 ≥95%（30 场景 × 3 次）	`quick_eval.py` 计数	Day 6 任务 1
S2	教学质量不掉太多	30 场景盲评，小模型总分均值 ≥ API 版的 85%（每场景 0-8 分制）	盲评表揭盲后统计	Day 6 任务 2
S3	延迟	p95 ≤ API 版的 1/2（同机同题；绝对值预期量级 10^2 ms）	时间戳实测 p50/p95	Day 4 基线 + Day 6 实测
S4	成本	单次调用成本 ≤ API 版的 1/10	卡时单价 × 单次秒数 vs API 单价 × token	Day 6 任务 1 的表
S5	通用能力不退化	第 6 周 20 题固定评测均分下降 ≤0.5（三轴 0-2, 满分 6）；且 GSM8K 子集 pass@1 不下降	同一份 20 题 + `gsm200` 前后各跑一次	Day 5 / 周 12
S6	CEFR 难度分类	分类准确率 ≥ 基座模型 + 8 个点（同 200 题标签集）	用 `reward_cefr` 的正确性判定当指标	Day 5 任务 3

完成判据：6 行无空格（**门槛栏不许出现「明显提升」「较好」这类词，必须可判卷**）；每行「怎么测」栏指向一个本周或周 12 会真实存在的脚本/表名；S3 与 S4 各写一句「基线数字来自 Day 4 实测，不是拍脑袋」。

> 注：门槛的**数字本身**今天允许写成区间或「预期量级」，Day 4 冻结协议时再定稿。但**结构不允许含糊**——S2 若写成「主观分不低于 API」，等于给自己留了临场解释的空间。**评测协议先于训练冻结**这条纪律的价值就在这里：门槛定完再训练，你才知道自己站在哪。

任务 3：范围与非目标（约 70 分钟）

1. 写死 **in-scope（三件事，不多不少）**：
   - T1 查词结构化释义生成（词性、例句、中文讲解放进固定 JSON）
   - T2 记忆口诀生成（明日记忆的主打能力，≤25 字）
   - T3 CEFR 难度分级（A1/A2/B1/B2/C1）+ 分级理由
2. 写死 **out-of-scope（明确不做，各写一句为什么）**：多轮长对话 tutor、语音/发音、个性化复习计划与 FSRS 调度、量化与端侧部署、上线灰度、SFT 之后的 DPO/RLVR（**周 11 的活**）、通用基准大规模跑分（**周 12 的活**）。
3. 一张「本周产出 → 后续周依赖」小图（画在 `PROPOSAL.md` 里）：本周 adapter + 冻结协议 → 周 11 复用周 8 的 DPO 管线与周 9 的验证器 → 周 12 接进明日记忆做真 A/B。

完成判据：in-scope 恰好 3 项且每项映射到 S1-S6 的某一条；out-of-scope ≥6 项且**每项都有「为什么不做」**（不许留空）；依赖小图存在。

> 注：范围这件事上，你的全栈背景反而是阻力——工程师的本能是「都能做，顺手加一个」。**M4 在简历上的价值来自闭环完整（数据→SFT→DPO→RLVR→评测→线上 A/B），不来自功能数量**。把 T1-T3 三项做到能报出数字，比做八项各报一个「有改善」强得多。

任务 4：三阶段路线与本周位置（约 60 分钟）

写清 M4 的三段（**本周一份、周 11 一份、周 12 一份，文档不许合并**）：

阶段	内容	复用哪周	交付	卡时预算
SFT（本周）	第 7 周教学数据 → LoRA 微调	周 6 管线 + 周 7 数据	adapter + A/B 基线 + 博客第 1 篇	≤14h ≈¥30
对齐（周 11）	DPO（口诀质量偏好对）+ RLVR（CEFR 分类 + 格式合法 + 释义事实性抽检）	周 8 管线 + 周 9 验证器	对齐效果对比表 + 博客第 2 篇	≤14h ≈¥30
评测与集成（周 12）	通用基准防退化 + 自建 200 题 + 明日记忆 vLLM 接入真 A/B + 模型上传	周 4/8/9 全套	可复现 repo + 博客第 3 篇 + 15 分钟讲稿	≤10h ≈¥30

完成判据：表三行齐全，「复用哪周」每格指向一个你实际存在的目录（不是概念）；三段卡时与简报 §5 的 P3 ≈40 卡时合计一致（14+14+10=38 ≤40）。

任务 5：风险清单与降级预案（约 60 分钟）

写五条，每条「触发条件 → 现象 → 降级动作」：

1. **训练不收敛 / eval loss 不降** → 按周 4 的顺序查 lr、数据管线、梯度流、初始化；降级：先把数据砍到 500 条跑通，再加回来。
2. **格式合法率上不去（S1 <80%）** → 这是**数据格式问题不是模型问题**：训练样本里 JSON 必须逐字一致（键序、缩进、是否允许多余文字）；降级：输出改单行紧凑 JSON，并在 prompt 里给一条完整示例（few-

shot 1 条）。

3. **1.5B 的盲评分大幅低于 API（S2 <70%）** → 属预期可能，写进「已知局限」；降级：不硬凑，改主张为「在高频子集（A1-B1 段）上达标」，范围收窄但结论做扎实。
4. **卡抢不到 / 显存爆** → 0.5B 先在本地跑完整流程，7B 不碰；seq 1024→512、`per_device_batch` 降、`gradient_accumulation` 升（等效 batch 不变）。
5. **API 余额不够跑基线** → 基线重复次数 5→2、场景数保持 30（**场景数是协议的一部分，绝不能砍；重复次数是测量精度，可以砍**）。

完成判据：5 条各有「触发 + 现象 + 动作」三段，第 2、3 条的动作具体到参数值或字段（**不是「优化一下」**）；第 5 条明确区分「协议量」与「测量精度」哪个能动。

任务 6：定稿入 repo 与收工（约 40 分钟）

```
mkdir -p E:/study/posttrain/llm-posttraining-lab/m4-capstone
cp E:/study/posttrain/week10/day1/PROPOSAL.md E:/study/posttrain/llm-posttraining-lab/m4-capstone/
cd E:/study/posttrain/llm-posttraining-lab && git add m4-capstone && git commit -m "M4: proposal, success criteria S1-S6" && git push
```

NOTES.md 三行：六条标准里你最没把握的那条 + 明天数据改造的第一个动作 + 「协议冻结」为什么要在训练前。

完成判据：`m4-capstone/PROPOSAL.md` 在 GitHub 可见（commit 今天）；NOTES 三行齐全。

今日验收：`PROPOSAL.md` 六节齐全（定位 / S1-S6 表 / 范围与非目标 / 三阶段路线 / 风险五条 / 依赖图）+ 已入 repo + 6 个面试问题列出且前两题有初步答案。

Day 2 • 周二：数据准备——复用第 7 周数据集 + 真实场景脱敏 + 任务格式

今日总览（6h）：M4 的数据来源已经在你手上（第 7 周发布的 `english-tutor-instruct`），今天做的是**把它改造成 tutor 任务格式**，并从明日记忆的真实日志里挑 **30 个查词场景。这 30 条今天就要封存，此后任何训练都不许见过它们**——它们是 Day 4 协议的地基。

任务 1：第 7 周数据集体检（约 70 分钟）

```
# week10/day2/audit_dataset.py — 体检报告，不许跳过
import json, collections, re
rows = [json.loads(l) for l in open("english-tutor-instruct.jsonl", encoding="utf-8")]
print("条数 =", len(rows), "| 字段 =", sorted({k for r in rows for k in r}))
print("主题 top8 =", collections.Counter(r.get("topic", "?") for r in rows).most_common(8))
lv = collections.Counter(str(r.get("cefr", "?")) for r in rows)
print("CEFR 覆盖 =", dict(lv), "| 未知 =", lv.get("?", 0))
lens = [len(r.get("output", "")) for r in rows]
print("输出长度 mean=%.0f p90=%.0f max=%d" % (sum(lens)/len(lens), sorted(lens)[int(.9*len(lens))],
max(lens)))
print("空字段 =", sum(1 for r in rows if not r.get("instruction") or not r.get("output")),
      "| 近重复 =", len(rows) - len({r.get("output", "")[:60] for r in rows}))
PII = re.compile(r"1[3-9]\d{9}|[\w.+-]+@[\w-]+\.[\w.]+|\d{17}[\dXx]")
print("PII 命中 =", sum(1 for r in rows if PII.search(json.dumps(r, ensure_ascii=False))))
```

完成判据：报告八行全部打印（**PII 命中必须 =0；不为 0 就停下，把命中条目贴进 gaps 并回查第 7 周的脱敏脚本，今天不许带着 PII 往下走**）；条数落在 2000-4000（第 7 周的量）；CEFR 未知标签占比 <20%（超过就把 T3 的范围缩到标签覆盖的子集，并写进 PROPOSAL 的范围一节）。

> 注：「近重复」那个口径是「输出前 60 字相同」，**它比第 7 周的 MinHash 粗得多，作用是给一个「不该太高」的哨兵**，别用它替代 MinHash。体检报告原样进 repo——**「我知道我的数据有什么问题」是后训练岗位的日常语言，比「我用了 3000 条数据」有力。**

任务 2：明日记忆真实日志脱敏 + 30 场景抽取（约 100 分钟）

1. 只读快照导入，**列白名单**：只接受「查询词 / 释义长度 / 时间戳」三列，其他列（含任何 id）在读取时直接丢弃：

```
# week10/day2/anonymize.py
import csv, json, re
ALLOW = {"word", "gloss_len", "ts"}                    # 白名单，不是黑名单
def clean(path):
    out = []
    with open(path, newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            r = {k: v for k, v in row.items() if k in ALLOW}
            if not re.fullmatch(r"[A-Za-z][A-Za-z\- ]{1,29}", r.get("word", "").strip()):
                continue                                # 只要纯英文词
            r["ts"] = r.get("ts", "")[:10]              # 日期留到天，不留时分
            out.append(r)
    return out
cleaned = clean("tutor_queries.csv")                    # Day 2 任务 2 步骤 3 要用
```

2. **分桶字段自己生成**，不许假设日志里有：`cefr_band` 用第 7 周数据里同词的标签（查不到就留 `NA`，NA 不进抽样）；`freq_band` 按词长 + 是否在数据集中出现做粗分（高频 = 在数据集里，低频 = 不在），并打印两档各多少条。

3. **PII 双保险**：正则扫（手机/邮箱/身份证/长数字）+ **人工抽检 40 条**（打印出来逐条看有没有「能认出是谁」的残留，例如某个自造词或专有名词），结论记「40/40 无风险」或列出问题。
4. 分层抽 30 个真实查词场景当 A/B 评测集（词频 high/low × CEFR 段），同时抽 **200 条**做 tutor 分类评测集（S6 用）：

```
import random
random.seed(2026)                                          # 写死：30 场景一旦定稿就不许换
buckets = [("high", "A1-A2", 8), ("high", "B1-B2", 7), ("high", "C1", 5),
           ("low",  "A1-A2", 4), ("low",  "B1-B2", 4), ("low",  "C1", 2)]
picked = []
for freq, lv, k in buckets:
    pool = [r for r in cleaned if r.get("freq_band") == freq and r.get("cefr_band") == lv]
    picked += random.sample(pool, min(k, len(pool)))
assert len(picked) == 30, f"只抽到 {len(picked)} 条，桶配额需调整并记录调整原因"
```

5. 落盘 `ab_scenarios.jsonl`（30 行）与 `cefr_eval200.jsonl`（200 行），**这两个文件从此只读**。

完成判据：`ab_scenarios.jsonl` 恰好 30 行（不足就**调整桶配额重抽**，不许为了凑数放宽分层，也不许让那条 assert 静默通过）；`cefr_eval200.jsonl` 200 行；PII 正则命中 0 + 40 条人工抽检结论写在 `day2/anon-check.md`；白名单写法是「只保留」而不是「删除已知敏感列」。

> 注：**这一步是整个 M4 唯一的合规命门**。真实产品日志即使已经匿名化，也可能有「一条自造短语就能定位到用户」的风险，所以正则 + 人工两道都要跑，且**人工抽检的 40 条要看内容不看字段名**。白名单优于黑名单：你不可能穷举所有敏感字段，但你能穷举你需要哪几列。写进 README 的话术：「日志只读导出三列，PII 正则扫描 0 命中，另做 40 条人工抽检」——这句在面试里的分量比模型分数重。

任务 3：tutor 任务的输出格式定义（约 80 分钟）

定一个**严格 JSON schema**（周 9 的 `reward_cefr` 与格式校验器直接接得上，周 11 的 RLVR 奖励就是它）：

```
# week10/day2/schema.py — 三任务共用一个外层信封
TUTOR_SCHEMA = {
  "type": "object",
  "required": ["word", "pos", "gloss_cn", "example_en", "mnemonic", "cefr", "reason"],
  "properties": {
    "word":       {"type": "string", "minLength": 1, "maxLength": 30},
    "pos":        {"type": "string", "enum": ["n.", "v.", "adj.", "adv.", "prep.", "conj.", "pron.",
"int."]},
    "gloss_cn":   {"type": "string", "minLength": 4, "maxLength": 60},
    "example_en": {"type": "string", "minLength": 8, "maxLength": 120},
    "mnemonic":   {"type": "string", "minLength": 4, "maxLength": 30},
    "cefr":       {"type": "string", "enum": ["A1", "A2", "B1", "B2", "C1"]},
    "reason":     {"type": "string", "minLength": 6, "maxLength": 80},
  },
  "additionalProperties": False,
}
if __name__ == "__main__":
    import json, jsonschema
    good = {"word":"reluctant","pos":"adj.","gloss_cn":"不情愿的、勉强的","example_en":"She gave a
reluctant nod.",
            "mnemonic":"「日落肯特」—太阳落下来了才「肯」，那就是不情愿。","cefr":"B2","reason":"常见但带情感
色彩的形容词，需语境理解"}
    jsonschema.validate(good, TUTOR_SCHEMA); print("SCHEMA SELFTEST OK")
    bads = [{**good, "cefr": "B3"}, {**good, "mnemonic": "x"*40},
            {k: v for k, v in good.items() if k != "reason"}, {**good, "extra": 1}]
    for b in bads:
        try: jsonschema.validate(b, TUTOR_SCHEMA); raise AssertionError("非法样本竟通过")
        except jsonschema.ValidationError: pass
    print("NEGATIVE CASES OK", len(bads))
```

完成判据：打印 `SCHEMA SELFTEST OK` + `NEGATIVE CASES OK 4`（非法档位 / 超长 mnemonic / 缺 required 键 / 多一个额外键，**四条必须全被拒**）；schema 进 repo 并在 PROPOSAL 的 S1/S6 栏被引用。

任务 4：切分 + 30 场景封存 + 收工（约 110 分钟）

1. 把第 7 周数据转成 tutor 任务格式（强模型改写 + 规则校验：转出的样本必须自己过 schema，**不过就丢**，别手改）。目标：T1/T2/T3 合计 2000-3000 条训练样本，存 `tutor_sft.jsonl`。
2. train/val/test 切分（90/5/5，**按 word 分组切**，同一个词的任何变体不许同时出现在 train 和 test）。
3. **封存 30 场景 + 200 CEFR 评测集**：跑 13-gram 去污染（复用周 8 的 `bad(x, y, n=13)`），确认训练集与之零交集，再落哈希：

```
import hashlib
h = hashlib.sha256(open("ab_scenarios.jsonl", "rb").read()).hexdigest()
open("ab_scenarios.sha256", "w", encoding="utf-8").write(h + "\n"); print("sha256 =", h[:16], "...")
```

4. NOTES.md 写：体检八个数字、PII 抽检结论、schema 四条负例、封存哈希前 16 位。

完成判据：训练样本 ≥2000 条且 **100% 过 schema 自校验**（打印通过数/总数两个数字）；按 word 分组的 train/test 交集打印为 0；训练集与 30 场景 + 200 评测集的 13-gram 命中数打印为 0；`ab_scenarios.sha256` 存在；NOTES 四项齐全。

> 注：**「按 word 分组切」而不是随机切**。同一个词的三个变体分别落到 train 和 test，你的 test 就在测「背没背过这个词」。**这条是第 8 周「同一 prompt 必须同侧」在实体维度的版本**——分组键要选业务上决定泛化的那一个。

今日验收：体检报告无 PII + 30 场景恰好 30 行 + schema 自校验与 4 条负例通过 + 训练集 ≥2000 且与评测集交集 0 + 哈希落盘。

Day 3 • 周三：训练方案设计 + 0.5B 冒烟

今日总览（6.5h）：把「怎么训」写成一页有依据的超参表，然后在 0.5B 上把整条管线（数据 → chat template → loss mask → 训练 → eval loss → 采样）**在本地跑通**。冒烟的目的不是分数，是**证明没有低级错**：模板拼对、mask 对、loss 真在降。

任务 1：模型规模选型（约 90 分钟）

三个候选的账算在 `day3/model-choice.md`，**用简报 §6 的量级公式手算（bf16：权重 2 字节/参、梯度 2 字节/参、AdamW 8 字节/参），不许写「大概」**：

候选	bf16 权重	LoRA 训练显存量级	单卡 4090 24G 结论	预期延迟量级（vLLM，≤300 token 输出）
Qwen2.5-0.5B	≈1.0GB	≈3-6GB（含少量梯度/优化器 + 激活）	极舒适，本地/单卡都行	10^1 ms 量级（最快）
Qwen2.5-1.5B（**主力**）	≈3.0GB	≈4-8GB（bf16 + gradient checkpointing + 1024 序列）	舒适，可 3 epoch 一天调两轮	10^2 ms 量级
Qwen2.5-7B-Instruct QLoRA	NF4 冻结基座 ≈3.5-4GB	+ LoRA 训练态 + 激活 ≈8-14GB	可行但慢，一轮实验以小时计	10^2-10^3 ms 量级

决策写死：**主力 1.5B LoRA**（质量与迭代速度的平衡），0.5B 做**冒烟与降级线**，7B-Instruct QLoRA 只在 Day 5 提前完成时作为**冲刺项**（且必须同步砍 Day 6 的重复次数）。

完成判据：表格四列 × 三行无空格；「结论」栏每格有明确判断（可行/舒适/慢）；决策三行写在表下，**并回答 Day 1 问题 2**（为什么不是 7B，要同时提到延迟与卡时预算两个约束）。

> 注：总计划正文写的是「3B/7B 开源模型」，本计划落到 **1.5B 主力 + 7B-Instruct QLoRA 冲刺**，理由是全程锁定的模型清单只有 0.5B/1.5B/7B-Instruct，且 4090 上 7B 全量单卡不可行、QLoRA 一轮实验按小时计，**撑不住「一天调一轮」的节奏**。这个偏离要在 PROPOSAL 里写一句说明——面试时「为什么从 3B/7B 改成 1.5B」是好问题，你的答案是预算与迭代速度，这比硬凑一个跑不动的 7B 有说服力得多。

任务 2：SFT 超参表（约 80 分钟）

项	值	依据（指向简报数字、上周实验或一句机制解释）
方法	LoRA（先）；7B 才用 QLoRA(NF4)	简报 §6：1.5B 全量在 4090 上优化器态即 ≈12GB + 权重梯度, 很紧
LoRA r / alpha	r=16、alpha=32（≈2r）	周 6 已用过；「为什么 rank 16」：任务格式固定，低秩够用，r 升 32 显存与过拟合风险都涨
target_modules	q/k/v/o + gate/up/down（7 个）	覆盖注意力 + FFN，输出格式主要靠 FFN 与投影层
lr	1e-4	简报 §6：LoRA 常 1e-4 ~ 2e-4
等效 batch	4（每卡）× 8（累积）× 1 卡 = 32	简报 §6 的估算式
epoch	3	数据 2-3k 条 → 预期量级 190-280 step
调度	warmup_ratio 0.03 + cosine	周 4 手写过的调度器，这里复用概念
max_length	1024	序列翻倍 ≈ 激活翻倍、attention 计算 ×4（简报 §6），1024 是 1.5B+4090 的舒适上界
精度	bf16 + gradient checkpointing	4090 优先 bf16（周 4 结论）；checkpointing 换显存
防遗忘 replay	混 5% 通用指令样本	周 6 灾难性遗忘 + 手册 Q7 的解法
seed	42（`seed` 与 `data_seed` 都设）	可复现

完成判据：11 行齐全且「依据」栏**每条都指向一个具体来源**；replay 那一行写明 5% 是**条数比例**还是**epoch 内混采**（二选一写死）；NOTES 里能默写等效 batch 算法。

> 注：DPO 的 lr（周 8 用的 5e-6）比这里 SFT 的 1e-4 低约 20 倍，**面试会追这个**：SFT 在学「格式与内容」，梯度信号直接（CE）；DPO 在学「谁比谁好」，信号是两句话的概率差，步子一大模型就去最大化 margin 而不是学偏好，表现为输出变长、语言退化。讲清楚这个，比背两个数字值钱。

任务 3：chat template 与 loss mask 落地（约 70 分钟）

先把 mask 可视化，再训练（周 6 的基本功，今天必须真做一次）：

```
# week10/day3/mask_check.py
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained("E:/study/posttrain/qwen05b")
messages = [
    {"role": "system", "content": "你是面向中国英语学习者的词汇教学助手，必须只输出规定的 JSON。"},
    {"role": "user", "content": "为 reluctant 生成结构化释义、例句、记忆口诀与 CEFR 难度。"},
    {"role": "assistant", "content": '{"word":"reluctant","pos":"adj.","gloss_cn":"不情愿的"}'},
]
full = tok.apply_chat_template(messages, tokenize=True, add_generation_prompt=False)
head = tok.apply_chat_template(messages[:2], tokenize=True, add_generation_prompt=True)
assert full[:len(head)] == head, "前缀不一致: BPE 把边界合并了, 见注"
labels = [-100] * len(full)
for i in range(len(head), len(full)): labels[i] = full[i]      # 只有 assistant 内容参与 loss
print("总 token =", len(full), "| 参与 loss =", sum(l != -100 for l in labels))
for i, (t, l) in enumerate(zip(full, labels)):
    print(f"{i:3d} {'LEARN' if l != -100 else '  mask'} {repr(tok.decode([t]))[:40]}")
```

完成判据：打印逐 token 的 LEARN/mask 列表；**system + user + 所有 ChatML 标记行都是 mask，只有 assistant 内容那段是 LEARN**；「参与 loss」的 token 数与单独 tokenize assistant 内容的 token 数一致（对照打印）。

> 注：那句 `assert full[:len(head)] == head` 上周出现过一次，今天会再出现：**BPE 会把 user 内容的最后一个字符和 assistant 起始标记合并成一个 token**，于是 head 不再是 full 的前缀，mask 边界整体错一格。真撞上就把边界那格也置 -100，**并在全部训练集上统计发生次数与占比**（Day 7 任务 1 就是这个统计）。别只是绕过去，要报出比例——这是「chat template 踩过什么坑」的标准答案格式。

任务 4：训练脚本落地 + 0.5B 冒烟（约 120 分钟）

```
# week10/day3/train_sft.py — 0.5B 本地冒烟 (Day 5 换 1.5B + 卡上路径)
import json, torch
from datasets import Dataset
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig
from trl import SFTTrainer, SFTConfig
from schema import TUTOR_SCHEMA
import jsonschema

MODEL = "E:/study/posttrain/qwen05b"
tok = AutoTokenizer.from_pretrained(MODEL); tok.padding_side = "right"   # 训练侧右填充，与 mask 一致
rows = [json.loads(l) for l in open("tutor_sft.jsonl", encoding="utf-8")]
ds = Dataset.from_list([{"messages": [
        {"role": "system", "content": r["system"]},
        {"role": "user", "content": r["instruction"]},
        {"role": "assistant", "content": r["output"]}]} for r in rows])
n = max(1, int(round(len(ds) * 0.9)))
train_ds, val_ds = ds.select(range(n)), ds.select(range(n, len(ds)))
print("train =", len(train_ds), "val =", len(val_ds))

model = AutoModelForCausalLM.from_pretrained(MODEL, torch_dtype=torch.bfloat16)
model.enable_input_require_grads()                       # LoRA + gradient checkpointing 需要
cfg = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                 target_modules=["q_proj","k_proj","v_proj","o_proj","gate_proj","up_proj","down_proj"])
args = SFTConfig(output_dir="E:/study/posttrain/week10/day3/smoke-out",
                 learning_rate=1e-4, per_device_train_batch_size=2, gradient_accumulation_steps=8,
                 num_train_epochs=1.0, max_length=1024, lr_scheduler_type="cosine", warmup_ratio=0.03,
                 bf16=True, gradient_checkpointing=True, logging_steps=5, seed=42, data_seed=42,
                 report_to="none", completion_only_loss=True)      # 见注：没有这个参数就自己写 mask
trainer = SFTTrainer(model=model, train_dataset=train_ds, eval_dataset=val_ds,
                     args=args, peft_config=cfg, processing_class=tok)
trainer.train(); trainer.save_model("E:/study/posttrain/week10/day3/smoke-adapter")
```

`python train_sft.py 2>&1 | tee smoke.log`

> 注：`completion_only_loss=True` 是较新版本 TRL 对 conversational 数据集的开关（**只对最后一段 assistant 算 loss**）。你的 trl 若没有这个参数：① `python -c "import inspect,trl; print(inspect.signature(trl.SFTConfig.__init__))"` 看有没有近义字段；② 没有就**自己写 mask**——labels 全填 -100，只在 `head_len..full_len` 填回 token id（任务 3 那段代码），并让 dataset 带 `labels` 列。**自己写反而是好事**：你会真正拥有这个知识点，而不是拥有调包能力。

完成判据：0.5B 冒烟跑满 ≥30 step（预期量级：2000 条 / 等效 16 ≈ 125 step/epoch，取 30 step 够判活）；**train loss 相对 step 5 的值下降 ≥20%**（打印首末两个 loss 数字）；`smoke-adapter` 目录非空；采样一条输出跑 `jsonschema.validate`，打印 `SMOKE JSON OK` 或明确的不通过原因。

> 注：**0.5B 冒烟的输出很可能是「合法 JSON 但内容是错的」**，这完全正常、也正是冒烟该看到的。冒烟判据里**不包含**内容质量；质量看 Day 5/Day 6 的 1.5B。把 0.5B 的烂输出截图存进 `day3/`，周 12 写「规模 vs 质量」对照时它是免费素材。

任务 5：方案文档（约 30 分钟）

`day3/train-plan.md`：模型选型表 + 超参表 + mask 结论 + 冒烟两个 loss 数字 + 明日「先冻结协议再训练」的一句话提醒。

完成判据：`train-plan.md` 五块齐全，且其中的超参表与任务 2 的表**逐行一致**（两份不一致是常见的隐藏 bug，明天 Day 5 就照这份跑）。

今日验收：model-choice 表 + 超参 11 行有依据 + mask 逐 token 打印 + 冒烟 loss 降 ≥20% + JSON 自校验有结论。

Day 4 • 周四：评测协议先于训练冻结（本周最重要的一天）

今日总览（6.5h）：在跑 1.5B 正式训练之前，把 A/B 协议写成文档、跑完 API 版基线、然后 git tag 钉死，此后不许改。这件事的工程名字叫「先写测试再写实现」，在后训练里的名字叫**防止你临场挑指标**。你的 Day 1 表格从「预期量级」升级为「基线实测后的定稿数字」，这是今天唯一的实质产出——但它决定 M4 全部后续结论的可信度。

任务 1：A/B 协议定稿（约 100 分钟）

`m4-capstone/ab-protocol.md` 必须写死以下 7 项（**每项一段，不许「另议」**）：

1. **题面**：`ab_scenarios.jsonl` 30 个真实查词场景，只读、哈希见 `ab_scenarios.sha256`。
2. **两个对比臂**：A = 现网 API 版（模型名、endpoint、温度、max_tokens 写死）；B = 小模型版（1.5B + 本方案模板，temperature=0.2、max_tokens=512、seed=42）。**两臂共用同一份 prompt 模板与同一个 schema**。
3. **重复次数**：每臂每场景 5 次（测量精度需要，非协议量；卡不够时降到 2，**必须在文档里写明降了**）。
4. **延迟定义**：从发请求到收到完整响应的墙钟时间，单位 ms；报告 **p50 与 p95**（不是平均，平均被 warm-up 污染）；**两侧各先跑 3 条 warm-up，不计入**。
5. **成本定义**：A = API 单价 × 平均 token 数（用返回的 usage）；B = 卡时单价 × 单次秒数（`¥/时 ÷ 3600`），**并注明 B 不含训练与显存摊销**（诚实边界，别把 B 算得过于便宜）。
6. **主观分**：任务 2 的 rubric，每场景 4 轴 × 0-2（0-8 分），盲评，每场景一次判定。
7. **判定规则（写死，不许训完再定）**：S2 通过 = B 均分 ≥ A 均分 × 0.85；S3 通过 = B 的 p95 ≤ A 的 p95 × 0.5；S4 通过 = B 单次成本 ≤ A 的 0.1；S1/S6 由脚本直接判。**任一「通过」为假都要照实写进结果表**，不许改门槛。

完成判据：7 项齐全、每项一段且含具体参数值；判定规则四条各带不等号与阈值；文档里出现「重复次数可降、场景数与门槛不可改」这句区分。

> 注：为什么 p95 而不是平均：查词是用户在等的交互，**尾部延迟决定体验**，而平均值会被 warm-up、首次 CUDA graph 捕获、冷 cache 拉偏。顺带一条：API 版延迟受网络与厂商排队影响，**同机测出来的比值偏乐观**（B 在本地、A 走公网），这个偏差必须在「诚实边界」里写一行，否则周 12 的 S3 结论会虚高。**主动写出对自己不利的偏差，是这份协议最值得被面试官看见的地方。**

任务 2：主观分 rubric 与盲评流程（约 80 分钟）

`m4-capstone/blind-rubric.md`：

轴	0 分	1 分	2 分
准确性	词义讲错或编造	大方向对但有不准确处	词义/词性/例句全对且无编造
教学可用性	看不懂在说什么	能用但绕	中国学习者一眼懂、例句自然
口诀助记性	无口诀或把释义重说一遍	有钩子但牵强	一眼能记住
简洁性	>60 字或含套话	31-60 字	≤30 字且无填充语

流程：同一场景的两臂输出打乱左右（`random.seed(2026)`），评分者看不到来源标签；**每场景 4 轴 × 0-2 = 0-8 分，30 场景合计 240 分满分，报告用每场景均分**；**必须同屏对比**，不许「先评完 A 再评 B」（跨批次记忆会污染）；全部评完揭盲；**随机抽 5 个场景揭盲后重评，两次判定不一致 ≥2 个则整轮作废重评**。

完成判据：rubric 4 轴 × 3 档无空格，其中「简洁性」的 2 分档写明确数字（≤30 字）；一致性规则（5 场景重评、≤1 不一致）写死；评分表模板 `blind-score.md` 已生成且含 30 行 × 4 轴空栏（**评分前生成，不许事后补表**）。

任务 3：API 版基线实测（约 90 分钟，先算账）

1. **先算账再跑**：30 场景 × 5 次 = 150 次调用，单次输入 ≈1.2k token（含模板）、输出 ≈0.5k → 约 0.18M 输入 / 0.075M 输出 token 量级；**用定价页实际单价乘一遍写进 NOTES**，>¥6 就先把重复次数降到 2（场景数不动）。
2. 跑基线并存全部原始响应（**延迟与 token 用量必须来自原始返回，不许事后凭印象**）：

```
# week10/day4/run_api_baseline.py
import json, os, time, statistics as st
from openai import OpenAI
cli = OpenAI(api_key=os.environ["LLM_API_KEY"], base_url=os.environ["LLM_BASE_URL"])   # 密钥只在环境变量里，别写进代码
SYS = open("system_prompt.txt", encoding="utf-8").read()
SC = [json.loads(l) for l in open("ab_scenarios.jsonl", encoding="utf-8")]
def call(word, n=512):
    t0 = time.perf_counter()
    r = cli.chat.completions.create(model=os.environ["API_MODEL"], temperature=0.2, max_tokens=n,
        messages=[{"role":"system","content":SYS},
                  {"role":"user","content":f"为 {word} 生成结构化释义、例句、记忆口诀与 CEFR 难度，只输出JSON。"}])
    return (time.perf_counter()-t0)*1000, r
for _ in range(3): call("example")                          # warm-up，不计入
rows = []
for s in SC:
    for k in range(5):
        ms, r = call(s["word"]); u = r.usage
        rows.append({"word": s["word"], "rep": k, "ms": round(ms, 1), "in_tok": u.prompt_tokens,
                     "out_tok": u.completion_tokens, "text": r.choices[0].message.content})
with open("api_baseline.jsonl", "w", encoding="utf-8") as f:
    for x in rows: f.write(json.dumps(x, ensure_ascii=False) + "\n")
lat = [x["ms"] for x in rows]
print("n=%d p50=%.0fms p95=%.0fms mean_in=%.0f mean_out=%.0f"
      % (len(lat), st.median(lat), sorted(lat)[int(.95*len(lat))-1],
         sum(x["in_tok"] for x in rows)/len(rows), sum(x["out_tok"] for x in rows)/len(rows)))
```

3. 顺手用 Day 2 的 schema 校验这 150 条响应，得到 **A 臂的格式合法率**（它会成为 S1 的对照）。

完成判据：`api_baseline.jsonl` 150 行（或降次后的实际数）；打印 n、p50、p95、平均输入/输出 token 四个数；A 臂 schema 合法率一个百分数（**必须实测得出，不许估算**）；NOTES 里有 API 花费的实际数字。

> 注：API 版 JSON 合法率通常**低于你的直觉**（模型爱在 JSON 前后加解释文字）。把这个数字放进结果表，它会自动成为 M4 立项最强的论据之一：**「输出必须被程序解析」这件事，通用 API + prompt 工程做不到 95%，微调过的 1.5B 能做到**——这就是「为什么要训而不是拼 prompt」的实证答案，比 Day 1 那段推理有力得多。

任务 4：协议冻结（约 50 分钟，硬性动作）

1. 把 S2/S3/S4 的门槛从「预期量级」升级为**基于 A 臂实测的定稿数字**（例如 S3 写成「B 的 p95 ≤ 实测 A_p95 × 0.5 = __ ms」，把那个数填进去）。
2. **冻结**：

```
cd E:/study/posttrain/llm-posttraining-lab
sha256sum m4-capstone/ab-protocol.md m4-capstone/blind-rubric.md data/ab_scenarios.jsonl > m4-capstone/frozen.sha256
git add -A && git commit -m "M4: FREEZE eval protocol before training (S1-S6 定稿)" && git tag m4-eval-protocol-v1 && git push --follow-tags
```

3. `PROPOSAL.md` 顶部加一行：「评测协议于 11/12 冻结（tag `m4-eval-protocol-v1`），早于 1.5B SFT 训练。」

完成判据：`git tag -l` 里有 `m4-eval-protocol-v1`；`frozen.sha256` 三行哈希；S3/S4 门槛栏填了**由 A 臂实测推出来的绝对数字**；这条 commit 的时间戳**早于** Day 5 的任何训练 commit（**以后要写进 README，它就是你没挑指标的证据**）。

> 注：**从今天起，改协议的唯一合法理由是被测对象出 bug（比如 schema 写错），且必须在文档里留下「第 X 条于 11/13 修改，原因 __，修改发生在第 1 轮训练之后」的字样**。不许悄悄改。这条自我约束就是本周存在的意义。

任务 5：通用能力基线 + 收工（约 70 分钟）

S5 的测量口子今天先打通（周 12 才正式跑分）：

1. 第 6 周那 20 道固定题，用今天的基线跑法各跑 1 次，记下 A 臂均分（三轴 0-2，满分 6）。
2. `gsm200` 上跑一次 1.5B **基座** pass@1（**temperature=0**），数字记下来——周 11 做完 RLVR 要拿它对照，别到时候找不到基线。
3. NOTES 三行：今天最险的一件事 / 明天训练前必须确认的三项 / 协议里你最担心撑不住的那条。

完成判据：两个基线数字进 `m4-capstone/baselines.md`（20 题均分 + GSM8K 子集 pass@1），各自写明测量配置（模型、温度、max_tokens、seed）；NOTES 三行齐全。

今日验收：协议 7 项 + rubric 四轴 + A 臂 150 行与四个数字 + **git tag 冻结** + S5 两个基线数字。

Day 5 • 周五：跑通第一版 SFT（1.5B LoRA）

今日总览（7h）：正式训练日。上午算账开卡，中午用 Day 3 冒烟过的脚本换 1.5B + 卡上路径跑 SFT，下午做 eval loss 体检与过拟合判断，晚上**用训好的模型把 30 个封存场景的输出采出来留档**（明天 A/B 的 B 臂素材就靠它，**今天必须产出**）。

任务 1：开卡算账 + 启动脚本（约 40 分钟）

```
cat > E:/study/posttrain/week10/day5/run.sh <<'EOF'
source /etc/network_turbo
pip install -U "trl>=0.11" "transformers>=4.45" peft accelerate wandb -i
https://pypi.tuna.tsinghua.edu.cn/simple
cd /root/autodl-tmp/week10 && python train_sft_15b.py 2>&1 | tee sft.log
EOF
```

预算写下：今日目标 5h 4090（¥8-12），本周累计上限 14h / ¥30；Day 3 若已用 1h，则今天 ≤5h、明天 ≤3h、留 5h 余量。**训练前先打印** `trl/transformers/torch` **三个版本**（实例上的版本与本地不一致是 Day 5 最常见的意外）。

完成判据：`run.sh` 已写好（**先写脚本再开卡，简报的记账纪律**）；NOTES 有「今日卡时预算 + 本周累计已用 + 剩余额度」三个数 + 实例上的三个版本号。

任务 2：1.5B LoRA SFT 训练（约 150 分钟）

把 Day 3 的 `train_sft.py` 只改五处（其余一字不动）：`MODEL` → `/root/autodl-tmp/qwen15b`；`per_device_train_batch_size=4`、`gradient_accumulation_steps=8`（等效 32，与 Day 3 超参表一致）；`num_train_epochs=3`；`output_dir=/root/autodl-tmp/out/tutor-sft-15b`；`report_to="wandb"`（连不上就 `export WANDB_MODE=offline`，回头 `wandb sync`）。

```
bash run.sh
nvidia-smi --query-gpu=memory.used --format=csv -i 0        # 每 20 分钟记一次峰值
```

完成判据：训练跑满 3 epoch（预期量级 2000-3000 条 / 等效 32 ≈ 190-280 step，以实际打印为准）；`train loss` 末值相对首个 logging 点下降 ≥40%；**eval loss 有值且末段不上扬**（上扬 → 过拟合，走任务 3）；adapter 目录非空；显存峰值 ≤20GB（截图 `day5/gpu.png`）。

> 注：**train loss 降而 eval loss 上扬 = 过拟合**，按简报 §6 的顺序处置：先加数据或加正则（dropout、减 epoch），**不是先去加参数**。tutor 格式极固定，1.5B LoRA 在 2000 条上 3 epoch 有过拟合空间——真出现了就把 epoch 降到 2，并记下「第几个 step 开始上扬」，这是周 14「失败案例」清单的成品一条。

任务 3：三方合法率与过拟合体检（约 60 分钟）

1. merge LoRA 权重（周 6 的 merge 流程）。**对同一份 `cefr_eval200.jsonl`，三个模型各生成一次输出**（基座 1.5B / 0.5B 冒烟 adapter / 1.5B SFT），分别落 `out_base.jsonl`、`out_05b_smoke.jsonl`、`out_sft15b.jsonl`（每行 `{"word":..., "text":...}`），然后统计合法率：

```
# week10/day5/quick_eval.py — 三方对照：基座 / 0.5B 冒烟 / 1.5B SFT
import json, jsonschema
from schema import TUTOR_SCHEMA
def legal_rate(path):
    rows = [json.loads(l) for l in open(path, encoding="utf-8")]
    ok = 0
    for r in rows:
        t = r["text"]
        try:
            jsonschema.validate(json.loads(t[t.index("{"): t.rindex("}") + 1]), TUTOR_SCHEMA); ok += 1
        except Exception: pass
    return ok, len(rows), round(ok / max(1, len(rows)), 4)
for p in ["out_base.jsonl", "out_05b_smoke.jsonl", "out_sft15b.jsonl"]:
    print(p, legal_rate(p))
```

2. 顺手用 `cefr_eval200` 的标签跑一次 CEFR 分类准确率（复用周 9 的 `reward_cefr` 判定），三方各一个百分数——**这就是 S6 的实测口径。**
3. 抽 5 条 SFT 前后同题输出并排贴进 `day5/compare.md`，人眼看两件事：**格式是否稳定**、**内容有没有复读训练集**（复读比 eval loss 上扬更早）。

完成判据：三方合法率六个数字（ok/n 与比例）+ 三方 CEFR 准确率三个数字打印入表；SFT 版合法率 > 基座版合法率（**SFT 最直接的证据，必须实测**）；`compare.md` 5 组对照存在且有一句「是否发现复读」的判断。

任务 4：30 场景 B 臂输出留档（约 100 分钟）

明天要盲评，**今天必须把 B 臂的 30 场景输出跑出来**，否则 Day 6 一早得先开卡（多花一次开卡与关机）。

```
# week10/day5/gen_b_arm.py — 卡上跑，与 Day 4 的 A 臂同模板
import json, time
from transformers import AutoModelForCausalLM, AutoTokenizer
M = "/root/autodl-tmp/out/tutor-sft-15b/merged"
tok = AutoTokenizer.from_pretrained(M)
model = AutoModelForCausalLM.from_pretrained(M, torch_dtype=torch.bfloat16).to("cuda").eval()
SYS = open("system_prompt.txt", encoding="utf-8").read()
rows = [json.loads(l) for l in open("ab_scenarios.jsonl", encoding="utf-8")]
with open("b_raw.jsonl", "w", encoding="utf-8") as f:
    for s in rows:
        msgs = [{"role":"system","content":SYS},
                {"role":"user","content":f"为 {s['word']} 生成结构化释义、例句、记忆口诀与 CEFR 难度，只输出
JSON。"}]
        ids = tok.apply_chat_template(msgs, return_tensors="pt", add_generation_prompt=True).to("cuda")
        t0 = time.perf_counter()
        g = model.generate(ids, max_new_tokens=512, do_sample=False, pad_token_id=tok.eos_token_id)
        txt = tok.decode(g[0][ids.shape[1]:], skip_special_tokens=True)
        f.write(json.dumps({"word": s["word"], "ms_hf": round((time.perf_counter()-t0)*1000, 1),
                            "text": txt}, ensure_ascii=False) + "\n")
print("done", len(rows))
```

完成判据：`b_raw.jsonl` 恰好 30 行（**与封存的 30 场景逐行核对 word 字段**）；每行 text 非空；打印 30 条里合法 JSON 的条数；文件回传本地。

> 注：这里 `do_sample=False`（贪心）与 A 臂的 `temperature=0.2` **不一致**，因为 HF 侧贪心最可复现。两种选择都合法，但**必须在协议文档里写明**；Day 6 用 vLLM 重测延迟时会用回 temperature=0.2 对齐 A 臂。**「哪里不一致、为什么、影响哪个结论」这三句话写清楚，就是评测工程的能力证明。**

任务 5：产物双备份、关机与收工（约 70 分钟）

```
cd /root/autodl-tmp/out && zip -r tutor-sft-15b.zip tutor-sft-15b
cd /root/autodl-tmp/llm-posttraining-lab && git add m4-capstone && git commit -m "M4: first SFT run" &&
git push
scp -P <端口> root@<实例>:/root/autodl-tmp/week10/b_raw.jsonl E:/study/posttrain/week10/day5/
```

完成判据：`git push` 成功且这条 commit 的时间戳**晚于** `m4-eval-protocol-v1`（先后顺序明天要写进 README）；`b_raw.jsonl`、`sft.log`、`gpu.png` 三件在本地各有一份；控制台状态「已关机」（**关机 ≠ 销毁**）；NOTES 记今日卡时与花费。

今日验收：3 epoch 跑完 + train/eval loss 两个趋势 + 三方合法率与 CEFR 准确率 + 30 行 B 臂输出 + 双备份与关机。

Day 6 • 周六：小模型版 A/B 实测 + 盲评 + 第 1 篇博客

今日总览（6h）：按 Day 4 冻结的协议，把 B 臂的延迟、成本、主观分测出来，揭盲、判 S1-S6，然后写 M4 系列第 1 篇博客。**今天最容易犯错的动作是「门槛没过就临场调门槛」**——协议已经钉死了，没过就写没过，这本身就是 M4 合格的产出。

任务 1：B 臂延迟与成本实测（约 90 分钟）

开卡 2-3h，用 vLLM 起服务（**这是周 12 直接入的预演**）：

```
vllm serve /root/autodl-tmp/out/tutor-sft-15b/merged --served-model-name tutor \
    --max-model-len 4096 --gpu-memory-utilization 0.85 --dtype bfloat16 --seed 42
```

```
# week10/day6/measure_b.py — OpenAI 兼容接口，与 A 臂同配置
import json, time, statistics as st
from openai import OpenAI
cli = OpenAI(base_url="http://127.0.0.1:8000/v1", api_key="EMPTY")
SYS = open("system_prompt.txt", encoding="utf-8").read()
SC = [json.loads(l) for l in open("ab_scenarios.jsonl", encoding="utf-8")]
def call(word, n=5):
    out = []
    for k in range(n):
        t0 = time.perf_counter()
        r = cli.chat.completions.create(model="tutor", temperature=0.2, max_tokens=512, seed=42,
            messages=[{"role":"system","content":SYS},
                      {"role":"user","content":f"为 {word} 生成结构化释义、例句、记忆口诀与 CEFR 难度，只输出
JSON。"}])
        out.append({"word": word, "rep": k, "ms": round((time.perf_counter()-t0)*1000, 1),
                    "in_tok": r.usage.prompt_tokens, "out_tok": r.usage.completion_tokens,
                    "text": r.choices[0].message.content})
    return out
for _ in range(3): call("example", 1)                          # warm-up，不计入
rows = [x for s in SC for x in call(s["word"])]
with open("b_arm.jsonl", "w", encoding="utf-8") as f:
    for x in rows: f.write(json.dumps(x, ensure_ascii=False) + "\n")
lat = [x["ms"] for x in rows]
print("B臂 n=%d p50=%.0fms p95=%.0fms mean_out=%.0f" % (len(lat), st.median(lat),
      sorted(lat)[int(.95*len(lat))-1], sum(x["out_tok"] for x in rows)/len(rows)))
```

成本一行写完：`B 单次 ≈ (p50 秒数) × (卡时单价 ÷ 3600)`；`A 单次 = API 单价 × 平均(in+out token)`。**两个数都从实测推出来，不许引用别人的数字。**

完成判据：B 臂 150 行（或降次后实际数）与 p50/p95 两个延迟数；S1 合法率（schema 校验 30 场景 × 3 次）；S6 与基座的分类准确率差值（一个百分点数字）；成本对比两行（A/B 各一行，含推导算式）。

> 注：今天的 p95 是**串行单请求**测出来的，真实服务是并发 batching，尾延迟通常更高——所以报告里必须写「单请求延迟」，**不许写成「线上延迟」**（周 12 接进产品才有后者）。另外 `vllm serve` 的参数名随版本变（部分版本用 `--gpu_memory_utilization` 下划线写法，`--seed` 在个别版本不接受），起不来先跑 `vllm serve --help | grep -iE "memory|seed"` 看当天那版怎么写，别照抄本文档硬试；不接受 seed 就去掉它并在 NOTES 记一句「B 臂未固定采样种子」——**这是协议的已知偏差，隐瞒比偏差本身严重**。

任务 2：30 场景盲评（约 100 分钟）

1. 用 Day 4 的模板生成同屏对比表（**左右顺序打乱**，标签只写「输出 1 / 输出 2」）：

```
# week10/day6/build_blind_sheet.py
import json, random
random.seed(2026)                                          # 写死：打乱结果可复现
A = [json.loads(l) for l in open("api_baseline.jsonl", encoding="utf-8")]
B = [json.loads(l) for l in open("b_arm.jsonl", encoding="utf-8")]
a1 = {}
for x in A: a1.setdefault(x["word"], []).append(x)          # A 臂每场景取 rep=0 那条
b1 = {x["word"]: x for x in B}
rows = []
for w in [json.loads(l)["word"] for l in open("ab_scenarios.jsonl", encoding="utf-8")]:
    side = random.choice(["ab", "ba"])                      # truth 列评分时不可见
    rows.append({"word": w, "left":  a1[w][0]["text"] if side == "ab" else b1[w]["text"],
                 "right": b1[w]["text"] if side == "ab" else a1[w][0]["text"], "truth": side})
with open("blind30.jsonl", "w", encoding="utf-8") as f:
    for r in rows: f.write(json.dumps(r, ensure_ascii=False) + "\n")
print("blind rows =", len(rows))
```

2. 按 Day 4 的 4 轴 × 0-2 逐条打分（**同屏比较，两侧各打分**）；30 条打完揭盲；抽 5 条重评查一致性（不一致 ≥2 则整轮重评）。
3. 统计：A 均分、B 均分、B/A 比值、胜/平/负计数、**四轴各自均分对比**（哪一轴 B 输得最多——这个信息比总分更能指导周 11 的对齐目标）。

完成判据：`blind30.jsonl` 30 行；评分表 30 行 × 4 轴 × 两侧全部有分（**空白 = 没评，不许交**）；S2 判定（`B/A ≥ 0.85`?）与胜平负三个计数；一致性抽查结论一行（5 条里几条不一致）；四轴对比表存在且指出最弱一轴。

> 注：盲评表必须由脚本生成、**评分前就存在**，且打乱的 `truth` 列不许被评分者看见。这份「表先于分数存在」的证据链和 Day 4 的 git tag 是一件事的两半——前者证明你没看答案再打分，后者证明你没看结果再定门槛。**周 12 README 把这两条并排列出，M4 的可信度就立住了。**

任务 3：结果表与 S1-S6 判定（约 60 分钟）

`m4-capstone/ab-result.md`：

标准	门槛（冻结值）	实测	判定	数据在哪
S1 格式合法率	≥95%			`b_arm.jsonl` 校验
S2 盲评	B ≥ A×0.85			`blind30.jsonl` 揭盲
S3 p95 延迟	B ≤ A×0.5			两侧实测
S4 单次成本	B ≤ A×0.1			算式 + 两个数
S5 通用能力	20 题下降 ≤0.5 / GSM8K 不降			Day 5 / 周 12
S6 CEFR	≥ 基座 +8 点			`quick_eval.py`

下面写三小节：**哪条最出乎意料**、**哪条你觉得测得不准以及为什么**、**周 11 该优先动哪一轴**（引用任务 2 的最弱轴）。

完成判据：六行全有实测数字与通过/未通过二值判定（**至少允许出现「未通过」，且未通过必须给 ≥3 行分析**）；三小节各 ≥4 行；表里没有一栏是空的。

> 注：n=30 的盲评均值**经不起细读**——一个场景从 8 分掉到 0 分就能把均分拉下约 0.27，而 S2 的门槛只差 15%。所以判完之后再加一列稳健对照：「去掉分数最极端的 1 个场景后重算」，两列并排写进表。**判定只用冻结口径那一列**，稳健列是给自己看的（周 11 决定动哪一轴时用它）。面试里能主动说出「我这个数字的样本量只够判 0.85 这条线，判不出 5% 的差距」，比把 30 场景的结果讲成结论可信得多。

任务 4：M4 系列博客第 1 篇（约 80 分钟）

标题建议：**《明日记忆 Tutor 小模型 • 第 1 篇：我把评测协议钉在了训练之前》**（M4 三篇中的第 1 篇，勿另开新篇；专栏名《从全栈工程师到后训练》）。1600-2400 字，六段：

1. 为什么一个全栈工程师要自己训模型（prompt 拼不出可解析的 JSON：引 A 臂合法率实测数字）。
2. 立项时定的六条标准，以及为什么每条都要带数字。
3. **协议先于训练冻结**：git tag + sha256 的截图，讲清它防的是什么（临场挑指标）。
4. 第一版 SFT 做了什么（数据来自第 7 周自己发布的数据集 + 真实日志脱敏；1.5B LoRA 超参表）。
5. 第一轮 A/B 结果（六个判定 + 一张盲评四轴对比图）。**过没过都说**。
6. 下一步（周 11 DPO + RLVR；一句带过，不展开）。

完成判据：博客已发布且链接可访问，链接贴进 NOTES 顶部；正文 ≥2 张图（四轴均分对比 + 任一其它图，例如 A/B 合法率条形图）；第 5 段**逐条列出 S1-S6 六个判定结果**（不许只报好消息）；第 3 段引用 tag 名与 commit 日期。

任务 5：周复盘（约 30 分钟）

`week10/day6/review.md` 五块：硬产出清单 / S1-S6 判定一句话 / 本周卡时与花费实际值 / 最不满意的一件事 / 周 11 第一件事。

完成判据：五块齐全，且「S1-S6 判定一句话」里**六条各出现一次**（漏一条 = 今天没读完结果表）。

今日验收：S1-S6 全判定 + 盲评表无空白 + 结果表三小节 + 博客第 1 篇已发布 + review.md。

Day 7 • 周日：补漏 + 闭卷复测 + 总验收 + 下周预习 + 求职节点

今日总览（4h）：不加实验。补两个洞，闭卷复测三道，7 项总验收，预习周 11 的对齐阶段，登记总计划里「第 10 周起可投日常实习」这个节点——**只登记，不投递**，最后做账面收口。

任务 1：补漏（约 80 分钟）

打开 `week10/gaps.md`。**最多补 2 个洞，贪多必失。** 优先这两类：

1. **mask 边界那格 / chat template 前缀不一致**：在全部训练集上跑一遍 tokenize，统计 assert 失败次数与占比；若 >0 就把边界格置 -100，重跑一次 Day 3 冒烟确认 loss 曲线没变差。
2. **S6 的 CEFR 只看了准确率、没看混淆矩阵**：打印 5×5 混淆矩阵，找出错得最多的那一档（**通常相邻档互混**，A2↔B1），写成周 11 的一条奖励设计输入。

其余洞原样交给后续周，今天不硬补。

完成判据：第 1 洞给一个**占比数字**；第 2 洞给一张 5×5 矩阵（numpy 打印即可）+ 一句「最混乱的是哪一档」。gaps 里标 `[x]` 并写「怎么补的」。

任务 2：闭卷复测（约 45 分钟，别跳）

白纸三题：

1. 写 SFT loss 与 label mask 规则，并画出「tutor 三消息」的 token 序列哪一段是 -100（**具体到 system/user/assistant 与 ChatML 标记**）。
2. 默写 M4 的六条成功标准与阈值：**其中哪两条的基线数字来自 Day 4 实测？为什么必须实测？**
3. 「你的 A/B 里，延迟与成本各是怎么算的？B 臂的成本算法里你故意没算哪两项？」（**答案：训练与显存摊销**）

完成判据：三题全答；第 2 题六条门槛报全且说得出「实测防的是临场改门槛」；第 3 题报得出两个未计入的摊销项。

> 注：第 3 题是本周唯一「只有诚实人才答得上」的题。把 B 臂成本算成「只用推理卡时」是好看的数字，但它会在面试第三层追问里炸。**主动说出自己结论里的乐观偏差，是后训练岗位最被看重的素质之一**——因为你每天要向工程与业务双方报告数字。

任务 3：总验收（约 30 分钟）

#	验收项	判据
1	立项	`PROPOSAL.md` 六节齐全；S1-S6 各有数字门槛 + 测量方法 + 产出脚本名
2	数据	训练样本 ≥2000 且 100% 过 schema；PII 正则 0 命中 + 40 条人工抽检结论；按 word 分组 train/test 交集 0
3	30 场景封存	恰好 30 行、`sha256` 已落盘、与训练集 13-gram 命中 0
4	方案与冒烟	超参表 11 行有依据；0.5B 冒烟 train loss 降 ≥20%；mask 逐 token 打印正确
5	**协议冻结**	tag `m4-eval-protocol-v1` 存在，`frozen.sha256` 三项齐全，commit 时间戳早于训练 commit
6	第一版 SFT	3 epoch 跑完；eval loss 未段不上扬；三方合法率对照（SFT > 基座）；B 臂 30 行输出留档
7	A/B 与输出	S1-S6 六行全判定（含未通过项的分析）；盲评表无空白且已揭盲；博客第 1 篇已发布

≥6 项通过 = 本周合格 = M4 的 SFT 阶段交付；不足 6 项，把缺的写进周 11 Day 1 前 2 小时（**但周 11 的对齐阶段与博客第 2 篇不许延期**）。

完成判据：7 行逐行填「通过 / 未通过 + 证据文件路径」，无空白格；第 5 行的证据必须是 `git tag -n` 与 `git log` 的两条真实输出（协议冻结早于训练），不接受口头声明。

任务 4：下周预习（约 30 分钟）

`notes/preview-week11.md` 写四条：

1. 周 11 = DPO + RLVR 移植到 tutor 数据。列出要复用的两个具体目录：`m2-sft/dpo/`（周 8 的标注与 `DPOTrainer` 脚本）与 `m3-grpo/`（周 9 的 `reward_cefr`、格式校验）。
2. 从今天的盲评结果挑出最弱那一轴，写一句「DPO 的偏好对 rubric 该重点打分哪一维」（**今天写，明天就不用重想**）。
3. 周 11 的 RLVR 三奖励 = CEFR 分类正确 + JSON 格式合法 + 释义事实性抽检。**第三项你还不知道怎么机械判定**，先写两个候选做法（例如「与权威词典 gloss 做词集重叠度」「规则校验例句含该词且长度合理」），明天验证哪个可跑。
4. 卡时预算：周 11 ≤14h ≈¥30，与本周持平。

完成判据：四条各 ≥3 行；第 3 条的两个候选各写一句「它可跑的前提是什么」。

任务 5：求职节点登记（约 35 分钟，只登记不投递）

总计划写着「第 10 周起即可投日常实习」。今天的动作严格限定为三件、共 35 分钟：

1. 把 M2/M3/M4 三行回填进简历 v2 的项目段草稿（**每行 = 一句话 + 一个数字**，例如 M4：「自建评测协议 + 1.5B SFT，JSON 合法率 **%、单次成本降至 API 的** 」）。
2. 收藏 10 个目标岗位 JD 存进 `jobs/shortlist.md`，分三档：大厂实验室 / 模型公司 / 业务侧 AI 团队；每档 ≥3 个，记录渠道。
3. **不做的事写死**：不投、不做简历终版、不约模拟面——这些在周 13（LeetCode 启动 + 简历终版 + 投递启动 + 第 1 次模拟面）。

完成判据：简历草稿有 3 行且各带一个真实数字；`jobs/shortlist.md` ≥10 条且分三档；文档里明确写了「投递启动在周 13」这一行。

> 注：为什么严格不许提前投：周 11-12 是 M4 的对齐与集成，**它决定你面试时能不能讲出完整闭环**；而面试一旦开始就会吃掉每周 6-10 小时（刷题、模拟面、复盘）。现在投出的每一个「先试试」，都要在周 15 的 ≥20 家集中投递里重新来过。**这份克制本身就是计划的一部分**，也是总计划把投递定在周 13 的原因。

任务 6：归档与账面收口（约 20 分钟）

账面四对写进 `week10/day7/review.md`：卡时上限 14h / 实际 **h；GPU ¥30 / 实际** ；API ¥6 / 实际 __；计划 40h / 实际 __h。并确认 `PROPOSAL.md`、`ab-protocol.md`、`ab-result.md`、`blind30.jsonl` 四件在 GitHub 与本地各有一份。

完成判据：四对数字齐全（**空着不许交**）；四件产物双备份路径列出；gaps 里每条都有归期。

今日验收：两个洞有产出 + 复测三题过 + 验收表 ≥6 项打勾 + 预习四条 + 求职登记三件（含不投递声明）+ 账面四对落纸。

本周暴露的洞 → 后续哪一周补

洞	归到	那天的动作
释义事实性无法机械判定（RLVR 第三奖励空缺）	**周 11 • Day 3**	落地词典重叠度 / 语法规则两个候选之一，做 30 条人工核对
CEFR 混淆矩阵只看相邻档，未做档位加权奖励	**周 11 • RLVR 奖励塑形**	把「相邻档 0.1」的塑形值改成基于混淆矩阵的自定义值
DPO 的偏好对 rubric 需按最弱轴重设权重	**周 11 • DPO 阶段**	复用周 8 管线，改 `score_prompt.txt` 权重后重跑 100 条复核
A/B 只有 30 场景 × 单 seed，方差未知	**周 12 • 自建 200 题 + LLM-as-judge**	同样本重跑 3 次做一致性抽查，把置信区间补上
B 臂成本未计训练/显存摊销	**周 12 • 成本对比表**	加一行「含训练摊销的全周期成本」，与 API 长期账单对照
通用基准只留了基线数字没跑分	**周 12 • lm-eval / OpenCompass**	`lm_eval --model hf --model_args pretrained=... --tasks gsm8k --batch_size auto` 正式跑防退化

三、常见问题与纪律

- **S1-S6 有没过的怎么办？ 照实写进 `ab-result.md` 与博客。** M4 的面试价值在「我知道自己成了什么、没成什么、为什么不」，不在全绿。周 15 项目深挖会问「哪条最险」，你的答案就是今天这个决定。
- **想临场改门槛？** 唯一合法理由是被测对象出 bug（schema 写错、模板不一致）。改之前先把原因写成一段并追加到 `ab-protocol.md` 的「修订记录」，**标注修订发生在第几轮训练之后**。不许静默修改。
- **某天没完成怎么办？** 不熬夜追。按「天-任务号」写进 `week10/gaps.md`，Day 7 是它们的位置。连续两天崩：先砍 Day 5 的 7B QLoRA 冲刺，再砍 Day 6 的重复次数，**Day 4 的协议冻结永远不砍**（砍了整周失去意义）。
- **卡住多久该求助？** 单点 40 分钟：写 NOTES → 搜报错原文 → 看对应框架仓库 issue → 再 20 分钟无解就降级（数据量减半、模型降到 0.5B、重复次数降），并在 gaps 立条目。**M4 的底线是「闭环跑通 + 协议在」，不是「分数漂亮」。**
- **钱与卡时**：卡时 ≤14h、目标 ≈¥30；API ≤¥6；全周上限 ¥36。Day 5 训练前先看本周累计已用。**超支的两个常见来源：Day 5 过拟合后重跑第三遍；Day 6 vLLM 服务起完忘了关实例。**每条都要在 NOTES 留下数字。
- **数据合规**：明日记忆真实日志走白名单三列 + 正则 + 40 条人工抽检；训练数据全部来自第 7 周你自己发布的公开数据集；**评测集 30 场景 + 200 条 CEFR 永不进训练集**（13-gram 命中必须为 0）。
- **别用 AI 代写立项文档**：允许它 review 你写好的 `PROPOSAL.md`、指出哪条标准不可判卷；**不允许**它替你决定成功标准——那六条数字是本周的训练目标，也是你面试要背的东西。